COMPUTER NETWORKS · BCSE308L / BCSE308P

ScamShield

Project guide and team study plan

Everything the three of us need to understand, present and defend the project: what it does, how every part works, the numbers to remember, how to run the demo, and who studies what.

Shreyans Chowdry
24BDS0114

Swapnil Jain
24BAI0248

Ayushmaan Nayak
24BCE2612

Vellore Institute of Technology · Repository: github.com/shreyans-chowdry/cn

CONTENTS

How to use this guide

Read chapters 1–3 together as a team first; everyone should be able to explain the project in two minutes. Then each person goes deep on the chapters listed in their role in chapter 8.

1. **The project in one page**: the problem, the three parts and the headline numbers
2. **How one SMS flows through the system**: an end-to-end walkthrough
3. **Detection concepts**: dataset, code-mixing, disguise levels, models, defences, metrics
4. **Networking concepts**: SSP, CRC and friends, ARQ, RTO, AIMD, the emulator, DNS, CIDR
5. **Results to remember**: one table of numbers per part
6. **Code map**: which file does what
7. **Running the demo**: laptop, phone and what to show
8. **Roles and presentation split**: who studies what and who presents which slides
9. **Viva question bank**: likely questions with short answers
10. **Glossary**

The two-minute pitch (everyone learns this)

> “Scam SMS for UPI come in many languages, mixed with English and each in its own script, and scammers disguise the words. We built a multilingual benchmark with five levels of disguise and showed that models that read characters hold up far better than models that read words. One of the best clean models, XLM-R, goes from missing 6% of scams to 52%. Our detector cleans the text first and reads characters, so it stays flat. We then deployed it as a networked system with our own CRC-protected protocol over TCP and reliable UDP, a DNS server and subnet checks on links, measured each mechanism, and built an Android app that warns you when a scam SMS arrives.”

The deliverables. The slide deck (20 slides), the IEEE paper (`ScamShield_IEEE_Paper.docx`) and this guide all use the same numbers, taken from `results/` in the repository. If a number here and a number in a slide ever differ, the repository is right.

CHAPTER 1

The project in one page

ScamShield detects UPI payment-scam SMS written in English or in another language mixed with English, even when the scammer disguises the words, and it delivers that detector as a real networked system. It is language-agnostic: a new language is added as data, not code.

The problem

Scam SMS ask you to “update KYC”, “enter your PIN to receive a refund”, or pay a small fee for a prize. Two things make them hard to filter:

- **They are multilingual.** People write their own language in its own script with English banking words mixed in, sometimes inside one word: आपका SBI account आज block कर दिया जाएगा। तुरंत KYC update करें; உங்கள் SBI account இன்று block செய்யப்படும், link-ஐ click செய்யவும்.
- **They are disguised.** Scammers write `P1N`, `K.Y.C`, a Cyrillic “е” inside “refund”, a letter from a neighbouring script that looks the same, or switch whole words into the other script, so filters that look for exact words miss them.

The three parts

1 · Detector and benchmark

UPI-MixSMS, an 11,979-message multilingual corpus; five disguise levels L0–L4; eleven models compared with significance tests; two defences; checks on real public SMS.

2 · Networked service

Our own protocol (SSP) with CRC-32 over TCP and over UDP with Go-Back-N / Selective Repeat and AIMD; a DNS client and server; CIDR checks on every link; each measured.

3 · Android app

Reads every incoming SMS, classifies it on the phone and notifies you of scams; can also ask the server over SSP. Same answers as the Python model.

Headline numbers

20 / 24
paired tests where character models degrade significantly less than word/subword models

6% → 52%
scams missed by XLM-R, clean → disguised; Char LR 10% → 19%

0.81 → 0.79
macro-F1 of the shipped detector, clean → hardest disguise; 0.878 on real smishing; ~1 ms

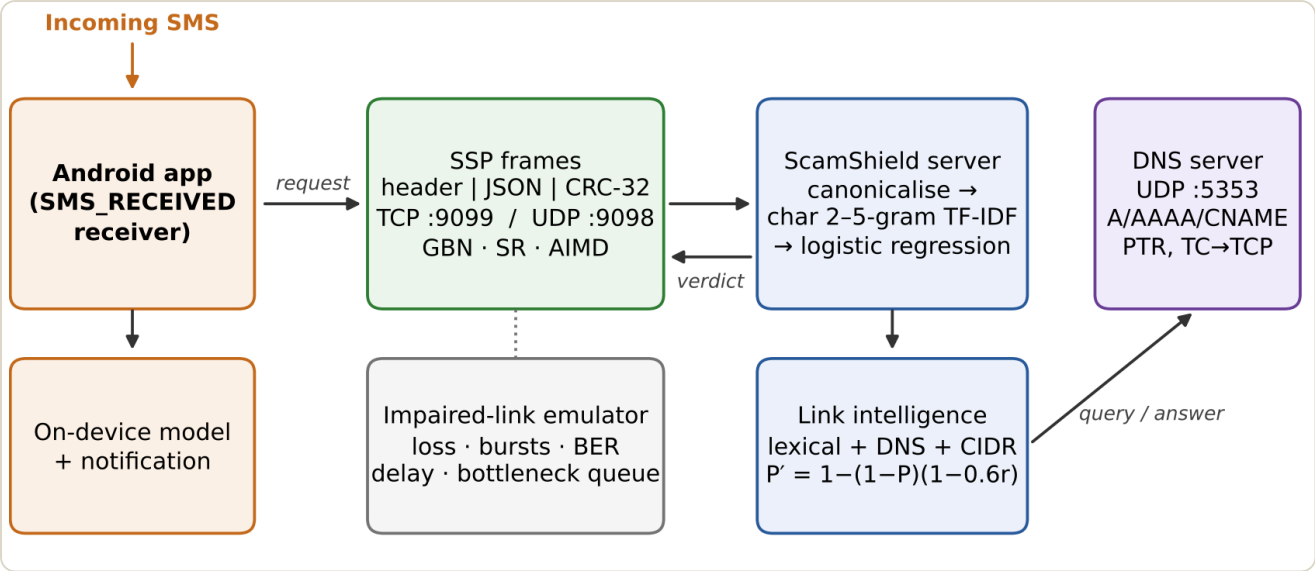
100%
corrupted frames caught by CRC-32

0.82 / 0.53
efficiency of Selective Repeat vs Go-Back-N at 10% loss

0
differences between phone and server answers on 300 test messages

CHAPTER 2

How one SMS flows through the system



The deployed system. Orange: the phone. Green: the protocol layer. Blue: the server and link checks. Purple: DNS. Grey: the emulator used only in experiments.

1. **The SMS arrives.** Android broadcasts `SMS_RECEIVED`; our `SmsReceiver` joins multi-part messages into one text.
2. **On-device check.** The Kotlin detector canonicalises the text, builds character 2–5-gram TF-IDF features, applies the logistic regression and looks at any links. If the verdict is “scam”, a high-priority notification appears (English and Tamil).
3. **Or server mode.** The phone wraps the text in an SSP `CLASSIFY_REQ` frame (16-byte header + JSON + CRC-32) and sends it over TCP port 9099. If the server is unreachable it falls back to step 2.
4. **The server checks the frame.** It reassembles frames from the TCP byte stream using the LEN field and recomputes the CRC; a corrupt frame is dropped.
5. **Text model.** Canonicalise (fold look-alikes, undo leetspeak and separators, map native-script loanwords back to English), then the character n-gram model gives P(scam), P(legit), P(promo).
6. **Link intelligence.** Each URL is resolved with our DNS client against our DNS server (UDP 5353). Lexical signs (shortener, risky TLD, brand name in a fake domain) and network signs (NXDOMAIN, very short TTL, address in a private or reserved CIDR block) give a risk *r* between 0 and 1.
7. **Fusion.** $P'(\text{scam}) = 1 - (1 - P(\text{scam}))\cdot(1 - 0.6\cdot r)$. Scam if $P' \geq 0.5$. Rule-based cues (“PIN to receive money”, “share OTP”, urgency, threats) are attached as human-readable reasons but never change the score.
8. **The verdict goes back** as a `CLASSIFY_RESP` frame; the phone shows it and notifies if needed.

> The same detector is also reachable over UDP port 9098 (single request/response, or our ARQ layer for batches) and through the web app on port 8000. All three entry points share one model.

CHAPTER 3

Detection concepts

3.1 The dataset: UPI-MixSMS

- **Three classes:** *scam*; *legit* (debit/credit alerts, OTPs, AutoPay mandates, real collect requests, payment chats, bank advisories, bill confirmations); *promo* (cashback, sales, loan offers, recharge, reward points).
- **23 intents**, 11 of them scams: KYC/PAN block, refund collect-request PIN fraud, lottery, "wrong transfer", job/task scams, electricity cut-off, customs parcel/FIR threats, fake customer support, loan fees, reward-point phishing, QR "scan to receive".
- **Hard negatives:** genuine messages reuse scam words ("do not share your OTP", advisories that mention KYC and links), so vocabulary alone cannot separate classes.
- **Multilingual by design:** English plus one code-mixed setting per supported language, each in its own script, with banking words kept in Latin script. Everything language-specific (templates, script tables, cue words) sits in one registry (`scripts.py`); adding a language is a data change. The current release has two code-mixed settings: Hindi–English in Devanagari (account में) and Tamil–English in Tamil script, where the suffix sits inside the word (link-ஐ). Romanised forms appear only as an attack (L4).
- **Size:** 11,979 messages (about 4,000 per setting), 3,939 test / 6,834 train / 1,206 validation. **Template-disjoint:** one template per intent is kept only for test, so scores measure generalisation to unseen phrasing.
- **Code-Mixing Index (CMI)** = 100 × (1 − largest-language tokens / language tokens). The two code-mixed settings average 39.0 and 40.6.
- **Automatic language ID** tags each word as English, native script, romanised native, *mix* (English stem + native suffix) or language-independent (numbers, URLs); 100% accurate on clean data. No user ever selects a language.

3.2 Disguise levels (obfuscation)

Level	p	Latin-script words	Native-script words (any registered script)	Char edits
L1 light	0.15	vowel drop, repeats, case, swaps, phonetic	confusable letters (श/ष/स, ಽ/ണ/ழ), vowel length, virama dropped/added	2.8%
L2 moderate	0.30	+ leetspeak (P1N), separators (K.Y.C)	+ separators (तु.रं.त)	7.8%
L3 heavy	0.45	+ Cyrillic/Greek look-alikes, zero-width chars, insert/delete	+ look-alikes from other blocks (त→ਰ, उ→ଉ), ZWJ, deletions	11.7%
L4 cross-script	0.60	+ English word in the native script (account → अकाउंट)	+ native word in Latin letters (turant)	27.4%

Fraud words (KYC, PIN, refund, तुरंत, इनाम, உடனே ...) are disguised with probability 2p. **Targeted threat:** only scams are disguised (the attacker only writes scams). **Uniform threat:** everything is disguised (tests false alarms on noisy genuine SMS). Three random seeds per condition.

3.3 Models: three ways of reading text

Family	Models	What they see
Word	Naive Bayes, Random Forest, Logistic Regression, SVM (word 1–2-grams); BiGRU	whole words; a single changed letter makes an unknown word
Subword	DistilmBERT, XLM-R, MuRIL	word pieces from a fixed vocabulary; disguised words shatter into odd pieces
Character	Char LR and SVM (char 2–5-grams), Char CNN	short character sequences; most of “acc0unt” still matches “account”

The **hypothesis**: character models lose less accuracy than word/subword models under character-level disguise.

3.4 Defences

- **Adversarial training:** half the training messages are replaced by L1–L3 disguised copies (L4 is never seen, to test generalisation).
- **Script-aware canonicalisation** (no retraining): NFKC; remove zero-width characters; fold Cyrillic/Greek look-alikes to Latin; fold native look-alikes (Gujarati letters, Tamil digits) back to the native letters; remove separators inside words; undo leetspeak; map native-script loanwords to English; squeeze repeats; lower-case. Cuts the remaining edit rate at L3 from 11.7% to 5.4%. It cannot undo native→Latin transliteration.
- **Shipped detector:** canonicalise → char 2–5-gram TF-IDF (60k features) → logistic regression (C = 10, balanced), trained on our corpus + disguised copies + real public SMS (Mishra & Soni, UCI, ExAIS genuine messages).

3.5 Metrics and statistics

- **Macro-F1:** the F1 of each class, averaged equally, so the small promo class counts as much as the others.
- **Scam FNR** (false-negative rate): share of scams missed. The most important number for users. **Scam FPR:** share of genuine/promo messages wrongly called scams.
- **ECE** (expected calibration error): does “90% sure” really mean right 90% of the time? 15 bins.
- **Paired bootstrap:** resample the test set 1,000 times and see how often the character model’s drop is smaller than the word model’s drop, giving a p-value. **McNemar’s test:** compares two models on the same messages using only the messages where they disagree.

CHAPTER 4

Networking concepts

4.1 SSP, the ScamShield Protocol (application layer)

MAGIC 2	VER 1	TYPE 1	FLAGS 1	WIN 1	SEQ 4	ACK 4	LEN 2	JSON payload LEN	CRC-32 4

Bytes per field, big-endian, packed with Python's `struct` format `!HBBBBIIH`. MAGIC is 0x5353 ("SS").

- **Frame types:** CLASSIFY_REQ, CLASSIFY_RESP, DATA, ACK, NAK, PING, PONG, ERROR, FIN.
- **Over TCP** (a byte stream with no message boundaries), the receiver buffers bytes, reads the 16-byte header, then waits for LEN + 4 more bytes. This is *framing* by a length field.
- **Over UDP** (datagrams), one datagram = one frame; reliability is our job (4.3).
- **Integrity:** the CRC-32 covers header and payload. Wrong CRC → the frame is silently dropped, and the sender's timer retransmits it.

4.2 Error detection and correction

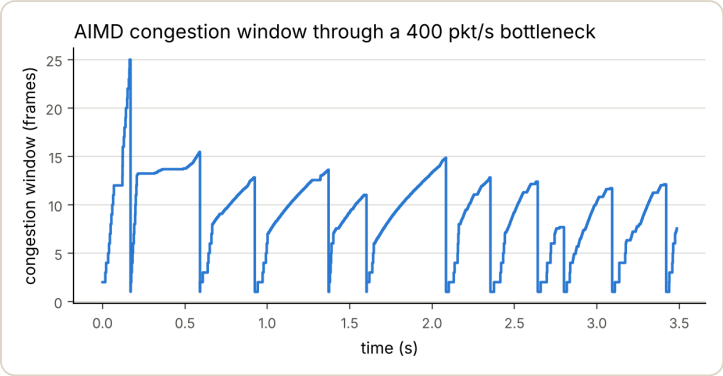
Code	Check bits	Catches	Misses (our measurement)
Even parity	1	any odd number of flipped bits	every even number of flips; ~50% of bursts
Internet checksum (RFC 1071)	16	most errors; one's-complement sum of 16-bit words	3% of random 2-bit errors (two flips that cancel in the sum)
CRC-16-CCITT	16	all bursts ≤ 16 bits, all odd errors	none observed
CRC-32 (IEEE 802.3)	32	all bursts ≤ 32 bits; polynomial division over GF(2)	none observed
Hamming(7,4)	3 per 4	corrects any single-bit error per 7-bit block	header recovered 94.5% at BER 10^{-2} (28.4% uncoded), +75% bits

4.3 Reliable delivery over UDP: ARQ

- **Stop-and-Wait:** send one frame, wait for its ACK. Simple, but throughput is limited to one frame per round trip (78 messages/s in our test).
- **Go-Back-N:** up to W frames in flight; the receiver accepts only in-order frames and sends a *cumulative* ACK; on timeout the sender resends *the whole window*. One timer.
- **Selective Repeat:** per-frame ACKs and timers; the receiver buffers out-of-order frames; only the lost frame is resent. Needs window ≤ half the sequence space.
- **Flow control:** the receiver advertises a window (WIN field) so a fast sender cannot overflow a slow receiver.
- **RTO (retransmission timeout), Jacobson/Karels:** SRTT ← 7/8·SRTT + 1/8·sample; RTTVAR ← 3/4·RTTVAR + 1/4·|SRTT − sample|; RTO = SRTT + 4·RTTVAR, at least 100 ms. **Karn's rule:** ignore RTT samples from retransmitted frames (you cannot tell which copy was ACKed). Timeouts double the RTO (exponential back-off); a new ACK resets it; 15 retries then give up.

4.4 Congestion control: AIMD

- **Slow start:** cwnd starts at 1 and grows by 1 per ACK (doubles per round trip) until ssthresh.
- **Congestion avoidance:** above ssthresh, cwnd grows by about 1 per round trip (additive increase).
- **On loss (timeout):** ssthresh ← cwnd/2, cwnd ← 1 (multiplicative decrease), at most once per window. The effective window is min(cwnd, receiver window).
- The result is the **sawtooth** in the figure below: AIMD finds the bottleneck's capacity by itself.



AIMD congestion window through a 400 packets/s bottleneck with a 12-packet queue.

4.5 The impaired-link emulator

A UDP proxy between client and server that can drop packets at random (**Bernoulli**) or in bursts (**Gilbert–Elliott**: a "good" and a "bad" state with different loss rates), flip bits (bit error rate), add delay and jitter while keeping order (FIFO), and act as a **bottleneck** with a limited service rate and a drop-tail queue. All experiments are seeded and repeatable.

4.6 DNS

- **Client** builds queries in the RFC 1035 wire format: 12-byte header (ID, flags, counts), question (name as length-prefixed labels, type, class), and parses answers including **name compression** (pointers starting with bits 11).
- Record types: A (IPv4), AAAA (IPv6), CNAME (alias), NS, PTR (reverse lookup via `in-addr.arpa`), TXT.
- **TC bit:** an answer over 512 bytes is truncated on UDP; the client retries over TCP with a 2-byte length prefix.
- **Caching:** answers kept for their TTL; NXDOMAIN cached too (negative caching). Cached lookup 0.20 ms vs 21.1 ms over the network.
- **Server:** authoritative for a demo zone using documentation address ranges; NXDOMAIN for anything else; chases CNAMEs.

4.7 IP addressing and CIDR

Every resolved address is matched (**longest-prefix match**) against the special-purpose registry (RFC 6890): private 10/8, 172.16/12, 192.168/16; loopback 127/8; link-local 169.254/16; CGNAT 100.64/10; reserved and documentation blocks; IPv6 unique-local fc00::/7 and link-local fe80::/10. A "bank" link that resolves into a private or reserved block is a strong scam sign that no text model can see. The web app also has a subnet calculator (network, broadcast, mask, host range, classful class).

CHAPTER 5

Results to remember

Detection: clean test set

Model	Type	Macro-F1	Scams missed	ECE	ms / message
MuRIL	subword	**0.872**	3.4%	0.116	108
XLM-R	subword	0.870	5.6%	0.132	97
Word NB	word	0.845	11.4%	0.137	0.6
Word LR	word	0.835	11.8%	0.071	1.4
Char LR	char	0.784	10.1%	**0.066**	1.5
DistilmBERT	subword	0.709	0.5%	0.229	65
Shipped detector	char + clean-up	0.810	9.5%	0.051	~1

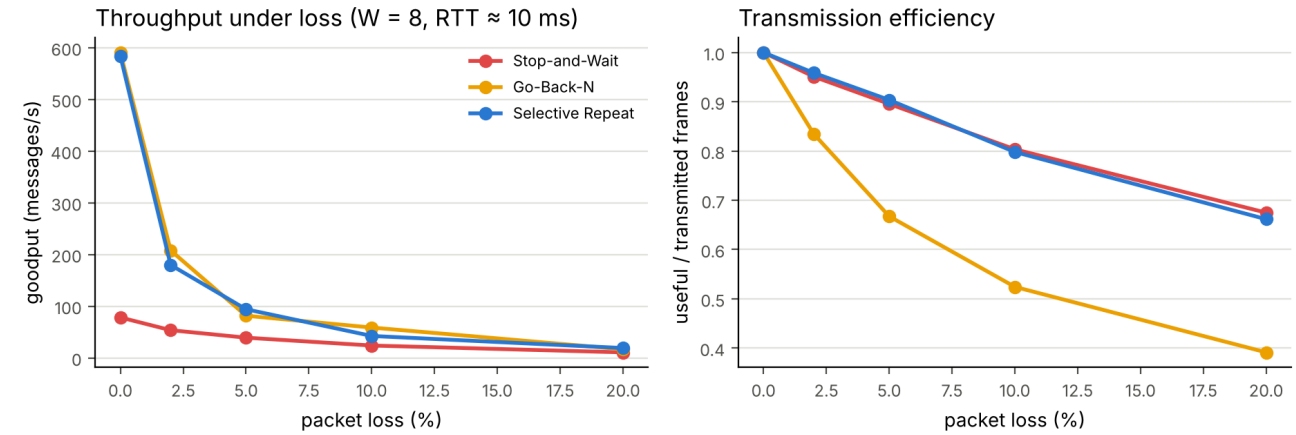
Detection: under disguise

- Uniform threat, L0 → L4 drop: XLM-R −0.255, MuRIL −0.178, Word LR −0.143, Char LR −0.097, Char SVM −0.081.
- Targeted threat, scams missed L0 → L4: XLM-R 5.6% → 51.5%; Word LR 11.8% → 27.9%; MuRIL 3.4% → 24.7%; Char LR 10.1% → 18.9%.
- Hypothesis: 20 of 24 paired tests at $p < 0.01$; 8 of 8 against the word models; 8 of 8 against XLM-R and MuRIL.
- DistilmBERT treats disguise as a sign of scam: it almost never misses a scam, but false alarms rise 25% → 41% under uniform disguise.
- Transfer between languages (train on one setting, test on the others): Char SVM loses 0.034, Char LR 0.049, word models 0.11, BiGRU 0.30.
- Canonicalisation alone: Word NB at L3 0.856 (vs 0.824 adversarial training, 0.803 neither).
- Shipped detector: 0.810 / 0.805 / 0.798 / 0.795 / 0.787 for L0–L4; 0.878 on real smishing (Mishra & Soni).
- Real data: our corpus only → 84% of real genuine SMS flagged; public only → 69–89% of UPI scams missed; both → works on both. Link checks do not change scams missed here (10.4%); they explain verdicts.

Network (N1–N6)

Experiment	Result
N1 error detection	CRC-32 / CRC-16: 100% detected. Parity misses all even errors and ~50% of bursts. Checksum misses 3.2% of 2-bit errors.
N2 ARQ under loss (W = 8, RTT ≈ 10 ms)	0% loss: GBN ≈ SR ≈ 590 msgs/s, SW 78. At 10% loss efficiency SR 0.80 vs GBN 0.52; at 20%: 0.66 vs 0.39.

N3 window size	37 msgs/s at W = 1 → 159–181 at W = 16–32 (filling the bandwidth–delay product).
N4 congestion (400 pkt/s, queue 12)	Fixed W = 48: ~1,040 drops, 61 msgs/s. AIMD: 120 msgs/s, 95% efficiency. W = 4 (matched): 198 msgs/s, no drops.
N5 TCP vs UDP	Transport only: UDP 0.28 ms, TCP 0.61 ms. With the model: ~3 ms either way (1.7 ms is the model); 290–360 msgs/s batched.
N6 DNS	Cached 0.20 ms vs 21.1 ms. TC → TCP fallback works. At 30% loss: 0/1/2/3 retries → 52/72/82/90% success.



N2: goodput and efficiency of Stop-and-Wait, Go-Back-N and Selective Repeat.

CHAPTER 6

Code map

File	What it does
`src/scamshield/scripts.py`	Registry of supported scripts: the one place a new language is added
`src/scamshield/data/templates*.py`	Parallel templates for the 23 intents, one list per language setting
`src/scamshield/data/generator.py`	Fills templates, adds natural variation, makes the template-disjoint split
`src/scamshield/data/public.py`	Loads Mishra & Soni, UCI, ExAIS and translated UCI; de-duplicates; grouped splits
`src/scamshield/hindi.py`, `tamil.py`, `langid.py`	Per-script tables (loanwords, confusables, look-alikes, romanisation); word-level language ID and CMI
`src/scamshield/obfuscation.py`	L1–L4 disguise operators and the canonicaliser
`src/scamshield/models/`	Word/char TF-IDF models, BiGRU, Char CNN, transformers
`src/scamshield/detector.py`	The shipped detector, link fusion, cues, export to the phone
`src/scamshield/experiments.py`, `analysis.py`	Runs every experiment; makes the tables and figures
`src/scamshield/net/protocol.py`	SSP frame encode/decode, TCP stream reassembly
`src/scamshield/net/errorcodes.py`	Parity, Internet checksum, CRC-16, CRC-32, Hamming(7,4)
`src/scamshield/net/arq.py`	Stop-and-Wait, Go-Back-N, Selective Repeat, RTO, AIMD
`src/scamshield/net/channel.py`	The impaired-link emulator (loss, bursts, bit errors, delay, bottleneck)
`src/scamshield/net/dns.py`, `cidr.py`, `urlintel.py`	DNS client/server; IPv4/IPv6 and CIDR; link risk scoring
`src/scamshield/net/service.py`, `client.py`	TCP and UDP servers; command-line clients
`src/scamshield/net/experiments.py`	Network experiments N1–N6
`app/main.py`, `app/static/`	FastAPI web app (four tabs); also starts SSP and DNS services
`android/app/src/main/java/com/scamshield/app/`	`sms/SmsReceiver.kt`, `detector/*.kt`, `net/Ssp.kt`, `ui/*.kt`
`scripts/train_production.py`, `run_all.sh`	Trains the shipped detector; runs the whole pipeline
`tests/test_core.py`	14 tests: data, obfuscation, CRC, Hamming, SSP, CIDR, DNS, ARQ

CHAPTER 7

Running the demo

Laptop (once)

```
cd ScamShield
./setup.sh          # creates .venv and installs requirements (one time)
./run_demo.sh       # web app http://localhost:8000
                    # SSP TCP 9099, SSP UDP 9098, DNS UDP 5353
```

Phone

1. Copy `releases/ScamShield.apk` to an Android 8+ phone and install it (allow “install unknown apps”; this is normal for an app not from the Play Store).
2. Open the app and grant SMS and notification permissions.
3. For server mode, put the phone and laptop on the same Wi-Fi, open the app’s **Network** tab, enter the laptop’s IP (shown on the web app’s Network tab) and port 9099, tap **Save and test**, then turn on **Use server mode**.

What to show, in order (about five minutes)

1. **Check a message.** Paste a code-mixed KYC scam (the “Code-mixed scam” sample), then try “Another script”. Point out the verdict, the reasons, the per-word language tags and the link card with its DNS answer.
2. **Disguise lab.** Raise the level from L0 to L4 and show where each model flips its answer while ScamShield holds.
3. **Network tab.** Build an SSP frame, press “Flip a random bit”, and show the CRC rejecting it. Resolve a suspicious link; try the subnet calculator.
4. **Phone.** On the Check tab, tap a sample such as “Code-mixed scam” and press “Simulate as incoming SMS”: the notification appears. Then a genuine OTP: no alert.
5. **Server mode.** Turn on server mode and repeat; the phone’s Network tab shows the request frame field by field and the round-trip time.
6. **Command line (optional).** `python -m scamshield.net.client batch messages.txt --arq sr --window 8` sends a batch over reliable UDP.

If something goes wrong

Symptom	Fix
Port already in use	Another copy is running: close it, or set `SCAMSHIELD_TCP` / `SCAMSHIELD_UDP` / `SCAMSHIELD_DNS` to other ports.
Phone cannot reach the server	Same Wi-Fi? College Wi-Fi often blocks device-to-device traffic: use a phone hotspot for both. The app falls back to on-device checks anyway.
Model file missing	`run_demo.sh` retrains the shipped detector automatically (about a minute).

CHAPTER 8

Roles and presentation split

Each person owns one part of the project: they study it in depth, present its slides and answer viva questions on it. Everyone still learns chapters 1–2 and the headline numbers.

Shreyans Chowdry · Project story, demo and integration

Slides 1–4 and 16–20 | Opens and closes the presentation | Runs the live demo

Owns

- The problem, the three parts and the overall story (chapter 1); the end-to-end flow (chapter 2).
- The dataset at overview level: classes, intents, the multilingual design and the script registry, template-disjoint split (3.1).
- The Android app as a user sees it, and why Android rather than iOS.
- Syllabus mapping and novelty; limitations and future work.
- The live demo (chapter 7) and keeping the final versions of the slides, paper and repository in sync.

Read

`README.md`, `paper/CN_MAPPING_AND_NOVELTY.md`, `android/README.md`, the web app's four tabs, chapters 1, 2, 5 (headline rows) and 7 of this guide.

Be ready for

Why is this a CN project? What is new? Why Android? Is the data real? How does the phone get the verdict? Who would use it? Hand detailed questions on models to Swapnil and on protocols to Ayushmaan.

Swapnil Jain · Detection models and results

Slides 5–11 | Answers questions on models and statistics

Owns

- The disguise levels and both threat models, with examples in Latin and native scripts (3.2).
- The eleven models, why character models should resist disguise, and how transformers tokenise (3.3).
- All detection results: robustness curves, scams missed, hypothesis tests, transfer between languages, defences, real-data experiments (chapter 5).
- Metrics and statistics: macro-F1, FNR/FPR, ECE, paired bootstrap, McNemar (3.5).
- The canonicaliser and the shipped detector (3.4).

Read

`paper/METHODS.md` sections 3–4 and 6, `paper/FINDINGS.md` sections 1–7, `results/RESULTS.md`, `obfuscation.py`, `detector.py`, `models/classical.py`; chapter 3 and the detection half of chapter 5.

Be ready for

Why does XLM-R fail under disguise when it is best on clean text? Why does DistilmBERT look robust on scams but raise false alarms? What does $p < 0.01$ mean here? Why macro-F1 and not accuracy? Why char 2–5-grams? What does canonicalisation not fix?

Ayushmaan Nayak · Network stack, protocol internals and on-device engineering

Slides 12–15 | Answers all protocol, DNS and Android-internals questions

Owns

- SSP byte by byte: header packing, TCP stream reassembly by LEN, UDP framing, frame types (4.1).
- Error control mathematics: parity, one's-complement checksum, CRC as polynomial division over GF(2) and why it catches all bursts up to its degree, Hamming(7,4) syndrome decoding (4.2).
- ARQ internals: Go-Back-N vs Selective Repeat state machines, window limits vs sequence space, Jacobson/Karels RTO, Karn's rule, back-off (4.3).
- Congestion control: slow start, congestion avoidance, multiplicative decrease, why AIMD converges, the sawtooth (4.4); the emulator and Gilbert–Elliott loss (4.5).
- DNS wire format, name compression, TC → TCP fallback, caching, PTR; CIDR longest-prefix match and special-purpose blocks; link-risk scoring and fusion (4.6–4.7).
- Experiments N1–N6 and how to read every network figure.
- Android internals: the SMS receiver, the Kotlin port of the detector and the parity test against Python; the public SMS corpora and how they are de-duplicated and split.
- The multilingual engineering: the script registry (`scripts.py`), the per-script tables, how the generic obfuscator and canonicaliser use them, and how the phone reads the scripts from `model.json` , so that adding a language needs no code change.

Read

`src/scamshield/net/` (every file), `paper/METHODS.md` section 5, `paper/FINDINGS.md` sections 8–9, `data/public.py` , `android/…/net/Ssp.kt` , `detector/Detector.kt` , `sms/SmsReceiver.kt` , `ParityTest.kt` ; chapter 4 and the network half of chapter 5. Kurose & Ross chapter 3 and Forouzan chapters 10–11 cover the theory.

Be ready for

Draw the SSP header. Why can Go-Back-N have a window of 2^n-1 but Selective Repeat only 2^{n-1}? Why does Go-Back-N waste frames at 10% loss? Derive the RTO formula. What happens on timeout in AIMD? Why does the Internet checksum miss some 2-bit errors but CRC does not? Walk through a DNS response with compression. Why is 10.0.0.5 suspicious for a bank link? How do you know the phone gives the same answer as the server?

Presentation order (about 15 minutes + demo)

Slides	Topic	Presenter	Time
1–4	Title, problem, what we built, dataset	Shreyans	3 min
5–11	Disguise, models, results, hypothesis, defences, real data	Swapnil	5 min
12–15	Architecture, SSP, network results, syllabus mapping	Ayushmaan	4 min
16–20	Android app, live demo, novelty, limits, Q&A	Shreyans	3 min + demo

CHAPTER 9

Viva question bank

Project and motivation

Is this really a Computer Networks project?
Yes. Besides the classifier it has an application-layer protocol with CRC, reliable transport over UDP (Go-Back-N, Selective Repeat), flow and congestion control (receive window, AIMD), DNS client and server, and CIDR addressing, all written from raw sockets and measured under loss, corruption and congestion. It covers lab experiments 2, 3, 4, 6, 8 and 9.

What is new compared to existing spam filters?
A multilingual UPI benchmark in native scripts that grows by adding data, attacks that work on any script including transliteration, a defence that needs no retraining, fusion of text with DNS/CIDR evidence, and an identical model on phone and server.

Why Android and not iOS?
iOS does not let third-party apps read incoming SMS, so automatic per-message warnings are only possible on Android.

Your data is synthetic. Why trust the results?
The test templates are never seen in training, and we also test on real public SMS corpora. Those experiments show that synthetic-only training fails on real SMS, which is why the shipped model is trained on both.

Detection

Why do character models resist disguise?
A disguise changes a few characters, so most character n-grams of the word survive; a word model sees an entirely new, unknown word, and a subword model sees unusual fragments.

XLM-R and MuRIL are most accurate. Why not ship them?
Under transliteration XLM-R misses 52% of scams (from 6%) and MuRIL 25% (from 3%); both are about 70× slower and less well calibrated. The shipped model goes from 0.81 to 0.79 across all levels in about 1 ms, and runs on the phone.

Why macro-F1 and the false-negative rate?
Classes are imbalanced, so accuracy hides failures on the small promo class; the false-negative rate is what hurts a user: a scam that gets through.

How would you add another language?
Write its parallel templates and one script module (Unicode range, confusable letters, look-alike characters, loanword spellings, romanisation rules), register it in `scripts.py`, regenerate the data and retrain. The obfuscator, canonicaliser, language identifier, web app and phone app all read the registry, so no other code changes.

What does the canonicaliser not fix?
Native words written in Latin letters (L4), because mapping them back needs a full transliteration model; that is why L4 remains the strongest attack.

Networking

How does the server know where one frame ends on TCP?
TCP is a byte stream. The receiver reads the 16-byte header, takes LEN, then reads LEN + 4 bytes (payload + CRC). Leftover bytes start the next frame.

Why is Go-Back-N less efficient than Selective Repeat under loss?
After a loss, Go-Back-N resends the whole window, including frames the receiver already got but discarded as out of order; Selective Repeat resends only the missing frame. At 10% loss: 0.52 vs 0.80.

Window size limits?
With n-bit sequence numbers, Go-Back-N can use $2^n - 1$; Selective Repeat at most 2^{n-1}, otherwise old and new frames with the same number become ambiguous.

How is the timeout chosen?
Jacobson/Karels: smoothed RTT plus four times its variation, at least 100 ms; doubled on each timeout; samples from retransmitted frames ignored (Karn).

Why did a fixed window of 48 do worse than AIMD?
It overflows the 12-packet bottleneck queue every round, causing about 1,040 drops and 500 retransmissions. AIMD backs off on loss and grows slowly, settling near capacity.

Why can CRC catch errors the checksum misses?
The checksum is a sum, so two flips that cancel go unnoticed. CRC is the remainder of polynomial division; any burst shorter than the generator's degree changes the remainder.

How does DNS help detect scams?
A link that does not exist (NXDOMAIN), has a very short TTL, or resolves into a private/reserved block cannot be a real bank site. These signals raise the link risk, which is fused with the text score.

What happens when a DNS answer is too big?
The server sets the TC bit on the UDP reply; our client repeats the query over TCP, where messages carry a 2-byte length prefix.

How do you know the phone gives the same answer?
A unit test runs 300 messages through the Kotlin detector and compares with Python: identical clean-up output, identical labels, largest probability difference 7×10^{-7}.

CHAPTER 10

Glossary

UPI	Unified Payments Interface, an instant account-to-account payment system; “collect requests” ask the payer to approve with a PIN.
Code-mixing	Using two languages in one message or one word (link-ஐ).
Transliteration	Writing a word in another script without translating it (account → அக்கவுண்ட்; உடனே → udane).
Homoglyph	A character that looks like another (Cyrillic “а” vs Latin “a”).
Virama	The mark that removes a consonant’s inherent vowel (् in Devanagari, the pulli ் in Tamil).
Template-disjoint split	Train and test sentences come from different templates, so the model cannot memorise phrasing.
n-gram	A sequence of n consecutive words or characters.
TF-IDF	Feature weighting: frequent in this message, rare across messages.
Macro-F1	Average of per-class F1 scores, each class weighted equally.
ECE	Expected calibration error: gap between confidence and actual accuracy.
ARQ	Automatic Repeat reQuest: reliability by acknowledgements and retransmission.
RTT / RTO	Round-trip time / retransmission timeout.
Goodput	Useful messages delivered per second (excluding retransmissions).
Efficiency	Useful frames divided by all frames sent.
cwnd / ssthresh	Congestion window / slow-start threshold.
Bandwidth–delay product	How much data fits “in the pipe”: link rate × round-trip time.
Drop-tail queue	A router queue that drops new packets when full.
CIDR	Classless addressing written as address/prefix-length (10.0.0.0/8).
NXDOMAIN	DNS answer meaning “this name does not exist”.
TTL (DNS)	How long an answer may be cached.